

Dune Console RBAC — Design & Architecture

yacketrj

2026-07-09

Dune Console RBAC — Design & Architecture

Table of Contents

1. Architecture Overview
 - 1.1 Dual-Boundary Model
 - 1.2 Boundary Rules
 - 1.3 Design Principles
2. Current State Analysis
 - 2.1 What We Have
 - 2.2 Gaps
3. Target Capability Model
 - 3.1 20 Capabilities — Grouped by Domain
 - 3.2 Capability Definitions
4. Tier-to-Capability Matrix
 - 4.1 Default Tier Assignments
 - 4.2 Owner/Admin Bypass
 - 4.3 Customization via Database
5. Web UI Authentication
 - 5.1 Login Flow
 - 5.2 Discord OAuth2 Configuration
 - 5.3 Discord API Calls
 - 5.4 Local Fallback
 - 5.5 CSRF Protection
6. Database Schema
 - 6.1 Table: dune.rbac_role_capabilities
 - 6.2 Table: dune.rbac_audit_log
7. Route Registration Pattern
 - 7.1 Current Pattern
 - 7.2 Target Pattern
 - 7.3 Migration Example
 - 7.4 Startup Validation
8. Capability Resolution Engine
 - 8.1 resolveSessionCapabilities(session, db)
 - 8.2 resolveTier(roleIds)
 - 8.3 Resolution Order (Priority)

- 9. Bot Integration
 - 9.1 Dual-Layer Enforcement
 - 9.2 Bot Config Update
- 10. Implementation Phases
 - Phase 1 — Extend Capability Model (1 session)
 - Phase 2 — Web UI Route Registration (2 sessions)
 - Phase 3 — Discord OAuth2 + DB-Backed Roles (3 sessions)
 - Phase 4 — Web UI Permissions Tab (2 sessions)
- 11. Migration Path
- 12. Key Decisions
- 13. Environment Configuration
 - 13.1 Required Environment Variables
 - 13.2 Optional Environment Variables
 - 13.3 Example .env
- 14. Sources & References

Dune Console RBAC — Design & Architecture

Version: 1.0

Date: 2026-07-09

Branch: integration/discord

Scope: dune-awakening-selfhost-docker (Core Fork) + dune-awakening-selfhost-discordbot (Discord Bot)

Upstream: [Red-Blink/dune-awakening-selfhost-docker](#)

Reference: [Icehunter/dune-admin](#) (capability model design)

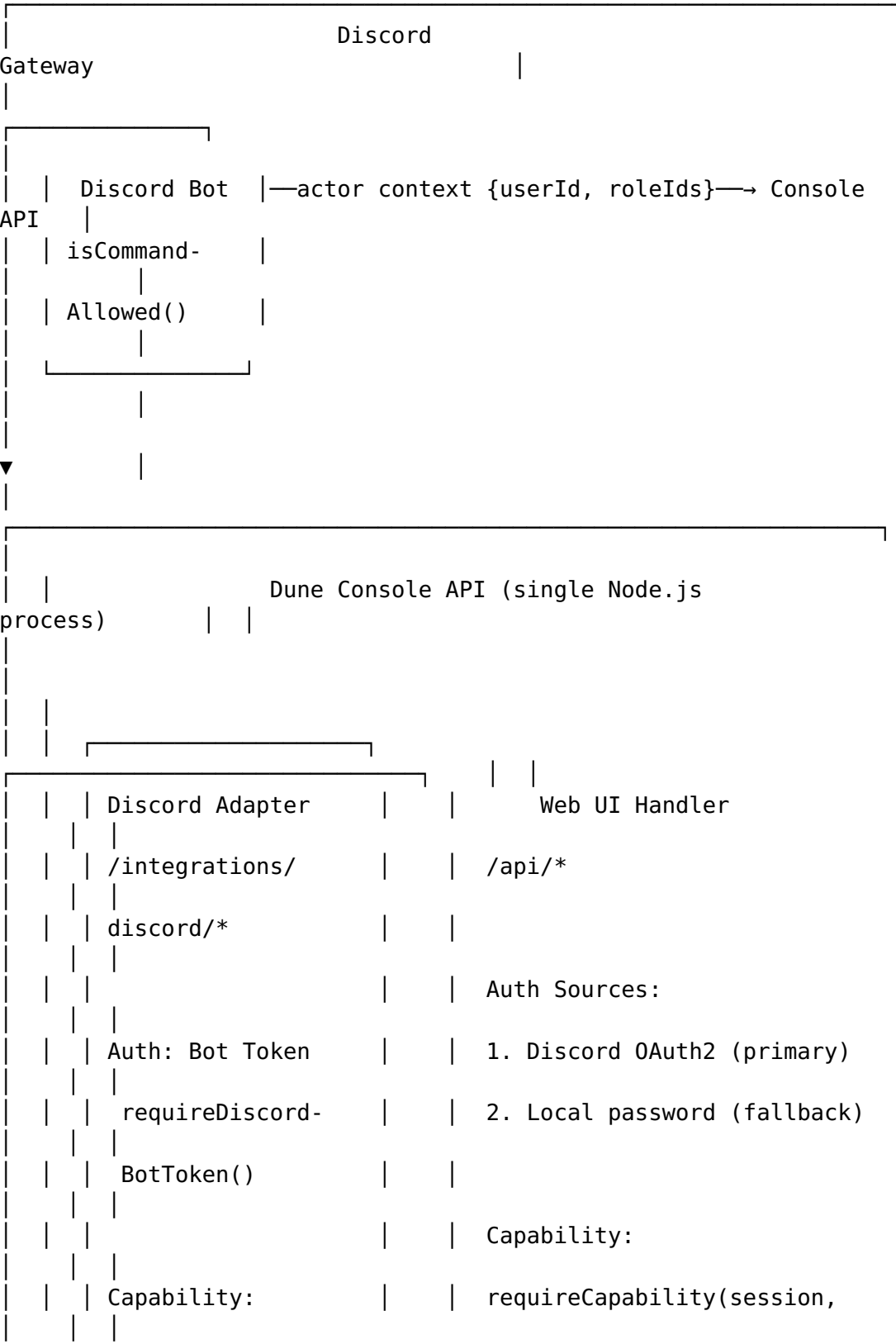
Table of Contents

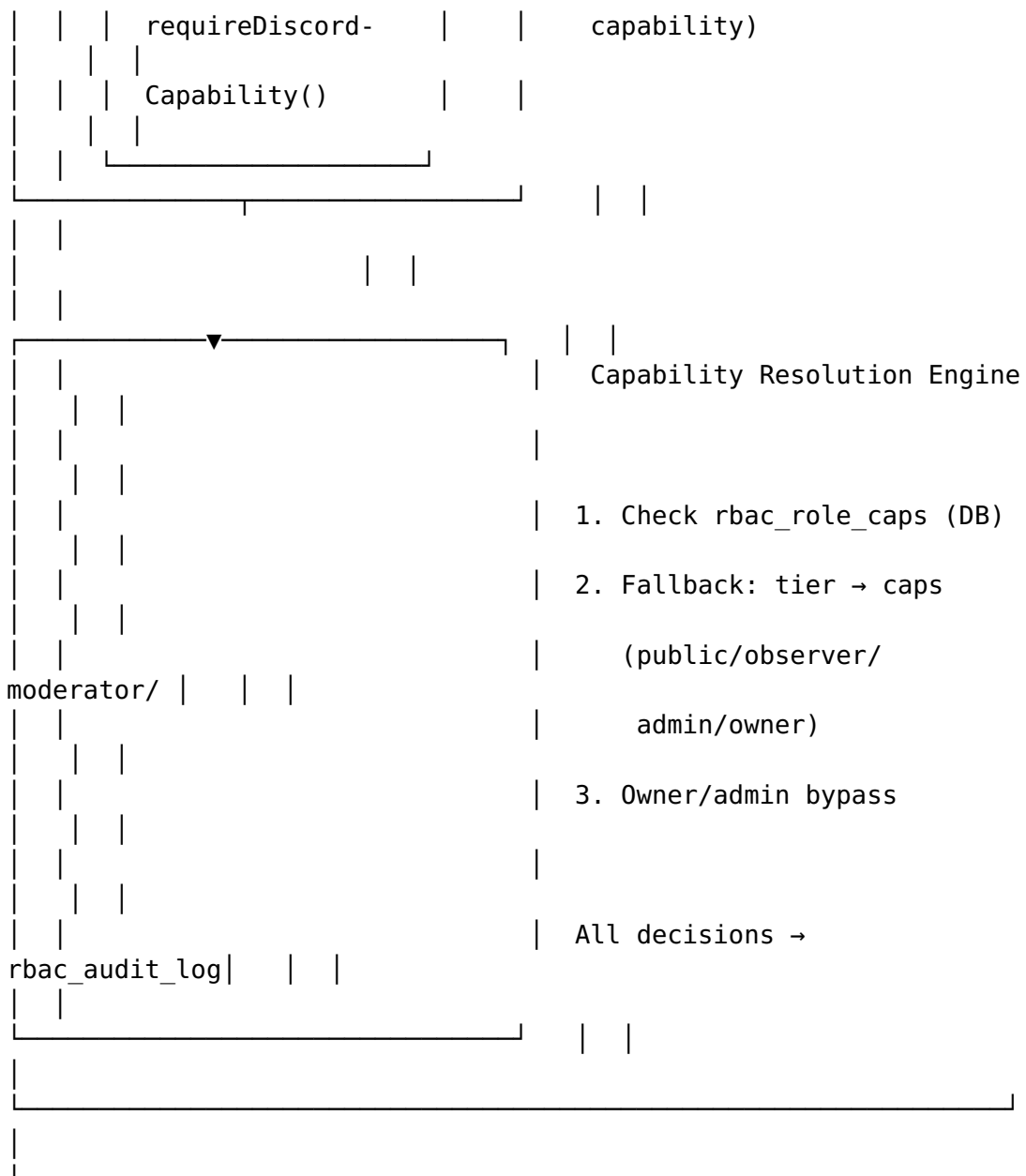
- 1. [Architecture Overview](#)
 - 2. [Current State Analysis](#)
 - 3. [Target Capability Model](#)
 - 4. [Tier-to-Capability Matrix](#)
 - 5. [Web UI Authentication](#)
 - 6. [Database Schema](#)
 - 7. [Route Registration Pattern](#)
 - 8. [Capability Resolution Engine](#)
 - 9. [Bot Integration](#)
 - 10. [Implementation Phases](#)
 - 11. [Migration Path](#)
 - 12. [Key Decisions](#)
 - 13. [Environment Configuration](#)
 - 14. [Sources & References](#)
-

1. Architecture Overview

1.1 Dual-Boundary Model

The Dune Console has two authentication surfaces that share a single capability resolution engine:





1.2 Boundary Rules

Boundary	Auth Mechanism	Capability Source	Fallback
Discord Adapter	Bot bearer token (DUNE_BOT_API_TOKEN_FILE)	Discord actor roleIds → tier → capabilities	Hardcoded tier mapping
Web UI	Session cookie (Discord OAuth2 or local password)	Session roleIds → tier → capabilities	Hardcoded tier mapping
Owner	Any auth path	ALL capabilities	Always bypassed

Boundary	Auth Mechanism	Capability Source	Fallback
Admin	Discord OAuth2 or local	ALL capabilities	Bypass via tier check

1.3 Design Principles

1. **Fail closed:** Missing capability = 403 denial, never silent success.
2. **Startup validation:** Every route must declare a capability at registration time. Missing capability is a programmer error — the server panics at startup, not at request time. (Inspired by dune-admin's `handleAPI()` pattern — see [Sources](#)).
3. **Belt and suspenders:** The bot gates commands client-side (`isCommandAllowed`). The adapter gates execution server-side (`requireDiscordCapability`). Both must pass.
4. **Backward compatible:** Empty role-capability table = existing hardcoded behavior. `DISCORD_OAUTH_CLIENT_ID` not set = local password only.
5. **Audit everything:** Every capability-gated action is logged with actor identity, target, and result.

2. Current State Analysis

2.1 What We Have

The Discord adapter already implements a tier-based capability model in `console/api/src/integrations/discord/policy.js`. It defines 10 read-only capabilities assigned to 5 tiers (public, observer, moderator, admin, owner). Every Discord adapter route calls `requireDiscordCapability(actor, mapping, capability)` which resolves the actor's role IDs to a tier, then checks if that tier includes the required capability.

The Discord bot (`dune-awakening-selfhost-discordbot`) has its own RBAC layer in `src/commands.js` via `isCommandAllowed()`, which checks the user's Discord role IDs against command-specific role allowlists in `config.js`.

2.2 Gaps

Aspect	Status	Gap
Discord adapter capabilities	10 read-only caps	Missing: <code>players:write</code> , <code>backups:manage</code> , <code>broadcast:send</code> ,

Aspect	Status	Gap
		server:control, database:query, database:read, diagnostics:read, guild:write, players:delete, players:read
Discord adapter enforcement	Tier-based (public→owner)	Works correctly for all registered routes
Web UI auth	Single admin password	No capability enforcement — all-or-nothing access
Web UI audit trail	Partial (admin-command-history.tsv)	No per-user audit, no capability-gated logging
Web UI user identity	None	Can't distinguish who performed an action
Bot RBAC	isCommandAllowed() via commandRoleIds	Works — no changes needed for Phase 1
Capability granularity	Tier-based only	No per-role customization; all moderators get same access

3. Target Capability Model

3.1 20 Capabilities — Grouped by Domain

Status & Health

#	Capability	Discord Bot Commands	Web UI Pages	Default Tier
1	status:read	core:about, core:ping, server:health, server:status, server:summary	Dashboard	public

#	Capability	Discord Bot Commands	Web UI Pages	Default Tier
2	readiness:read	server:readiness	Readiness panel	observer
3	services:read	server:services, infra:version, infra:servers, infra:ports, infra:db	Services panel, container list	observer

Population & Maps

4 | population:read | data:population | Player count | moderator |
5 | maps:read | data:maps | Live map, map state | moderator |

Logs & Diagnostics

6 | logs:read | — | Log viewer | admin |
7 | diagnostics:read | admin:doctor, admin:latency, admin:cooldowns, admin:events | Diagnostics panel | admin |

Database & Backups

8 | backups:read | data:backups | Backup list, download | moderator |
9 | backups:manage | — | Backup create, restore, delete | owner |
10 | database:read | — | Table preview, schema browser | admin |
11 | database:query | — | SQL query, database search | owner |

Player Management

12 | players:read | data:whoami | Player list, profile, inventory viewer | moderator |
13 | players:write | — | Give items, XP, currency, augments, teleport | admin |
14 | players:delete | — | Character delete, clean inventory, account delete | owner |

Inventory & Storage

15 | inventory:read | data:inventory, data:inventory search:<item> | Inventory viewer | moderator |
16 | storage:read | data:storage, data:find | Storage browser, item search | moderator |

Guild & Social

17 | guild:read | data:storage scope:guild, data:find scope:guild | Guild list, guild inventory | moderator |
18 | guild:write | — | Edit guild description, manage members | admin |

Server Control

19 | server:control | — | Start/stop services, restart, update, spawn vehicles | owner |
20 | broadcast:send | admin:broadcast | Broadcast panel | admin |

3.2 Capability Definitions

Each capability has a human-readable description used in the Permissions tab and audit logs:

Capability	Description
status:read	View server health, status, and adapter metadata
readiness:read	View server readiness and preflight state
services:read	View service containers, infrastructure routes, and version
population:read	View player population counts
maps:read	View active game maps, sietch, and deep desert state
logs:read	Stream and view capped, redacted server logs
diagnostics:read	View system diagnostics, latency history, cooldowns, incident log
backups:read	List and download backup metadata
backups:manage	Create, restore, and delete database backups
database:read	Browse database tables and schemas
database:query	Execute read-only SQL queries and export data
players:read	View player list, profiles, and inventory
players:write	Give items, XP, currency, augments, teleport players
players:delete	Delete characters and wipe inventories

Capability	Description
inventory:read	View your linked character's personal inventory
storage:read	View owned and guild storage containers, search items
guild:read	View guild listings and member directories
guild:write	Edit guild descriptions and manage member roles
server:control	Start/stop services, trigger updates, spawn vehicles
broadcast:send	Send in-game broadcasts to all players

4. Tier-to-Capability Matrix

4.1 Default Tier Assignments

These are the hardcoded fallbacks when `dune.rbac_role_capabilities` is empty (existing installs):

public — status:read

observer — + readiness:read
+ services:read

moderator — + population:read
+ maps:read
+ backups:read
+ players:read
+ inventory:read
+ storage:read
+ guild:read

admin — + logs:read
+ diagnostics:read
+ database:read
+ players:write
+ guild:write
+ broadcast:send

owner — + backups:manage
+ database:query
+ players:delete
+ server:control

4.2 Owner/Admin Bypass

Owners and admins receive ALL 20 capabilities automatically. No explicit capability assignment is needed. This follows dune-admin's convention where owners bypass the entire capability matrix.

The bypass is enforced in two places: 1. **Session creation:** When a user authenticates with a role matching `DISCORD_OWNER_ROLE_IDS` or `DISCORD_ADMIN_ROLE_IDS`, their session is stamped with `ALL_CAPABILITIES`. 2. **Local fallback:** When the user authenticates with `ADMIN_PASSWORD`, their session is stamped with the owner tier and `ALL_CAPABILITIES`.

4.3 Customization via Database

When `dune.rbac_role_capabilities` has rows, the DB takes precedence over the hardcoded tier matrix. An operator can:

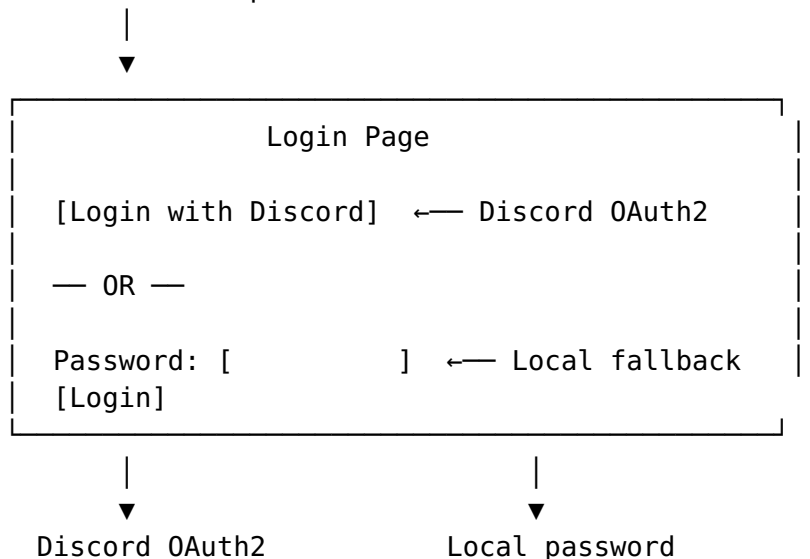
- **Add** capabilities to a role beyond its tier default (e.g., give a specific moderator role `players:write`)
- **Remove** capabilities from a role (e.g., restrict an admin role from `server:control`)
- **Create** custom Discord roles with any capability combination

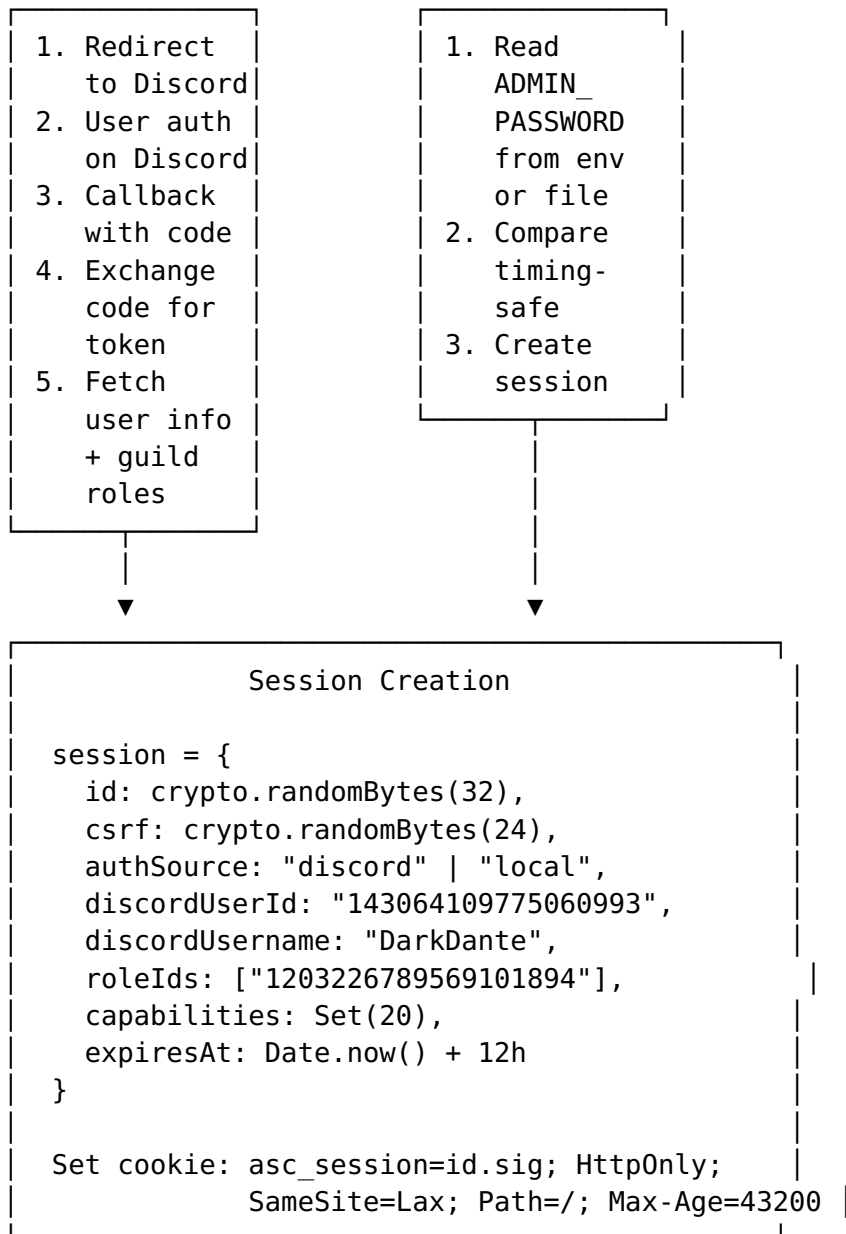
Owners and admins are exempt from DB-based restrictions — they always receive all capabilities.

5. Web UI Authentication

5.1 Login Flow

User visits `http://console:8088`





5.2 Discord OAuth2 Configuration

New env vars for Discord OAuth2 (Web UI login)

DISCORD_OAUTH_CLIENT_ID=1516816812006969494

DISCORD_OAUTH_CLIENT_SECRET=<discord-app-client-secret>

DISCORD_OAUTH_REDIRECT_URI=http://50.123.64.61:8088/api/auth/discord/callback

Role-to-tier mapping (shared with Discord bot)

DISCORD_OWNER_ROLE_IDS=1203226789569101894

DISCORD_ADMIN_ROLE_IDS=1203226789569101894

DISCORD_MODERATOR_ROLE_IDS=1207762798705123398

DISCORD_OBSERVER_ROLE_IDS=1203226789569101894,1207762798705123398

5.3 Discord API Calls

The OAuth2 flow uses standard Discord endpoints (see [Discord OAuth2 Documentation](#)):

```
GET https://discord.com/api/oauth2/authorize
    ?client_id=<CLIENT_ID>
    &redirect_uri=<REDIRECT_URI>
    &response_type=code
    &scope=identify%20guilds
    &state=<crypto.randomBytes(16).toString('hex')>
```

```
POST https://discord.com/api/oauth2/token
    Body: client_id, client_secret, grant_type=authorization_code,
    code, redirect_uri
```

```
GET https://discord.com/api/users/@me
    Authorization: Bearer <access_token>
```

```
GET https://discord.com/api/users/@me/guilds
    Authorization: Bearer <access_token>
```

The guilds endpoint is called to verify the user is a member of the configured guild (DISCORD_GUILD_ID). The user's roles in that guild determine their capability set.

5.4 Local Fallback

The existing ADMIN_PASSWORD mechanism remains unchanged. This is the emergency access path:

- **When:** DISCORD_OAUTH_CLIENT_ID is not configured, OR the user chooses local password login
- **Auth:** timingSafeEqual(input, config.adminPassword)
- **Session:** Stamped with authSource: "local", owner tier, ALL capabilities
- **Audit:** Logged as actor_id: "local:admin"

When ADMIN_AUTH_DISABLED=1, both Discord OAuth2 and local password are skipped — full access for development.

5.5 CSRF Protection

The state parameter in the OAuth2 flow prevents CSRF attacks:

```
// Store state in a short-lived Map
const oauthStates = new Map(); // state → { expiresAt,
    redirectTo }
```

```
// Generate state before redirecting to Discord
const state = randomBytes(16).toString("hex");
```

```

oauthStates.set(state, { expiresAt: Date.now() + 300_000,
    redirectTo: originalUrl });

// Verify state on callback
function verifyOAuthState(state) {
    const entry = oauthStates.get(state);
    oauthStates.delete(state);
    if (!entry || entry.expiresAt < Date.now()) return null;
    return entry;
}

```

6. Database Schema

6.1 Table: dune.rbac_role_capabilities

Stores per-role capability assignments. Rows in this table **override** the hardcoded tier defaults. An empty table = system defaults.

```

CREATE TABLE IF NOT EXISTS dune.rbac_role_capabilities (
    role_id    TEXT NOT NULL,          -- Discord role snowflake ID
    capability TEXT NOT NULL,          -- e.g., "players:write",
    "backups:manage"
    granted_by TEXT,                   -- who granted (Discord user
    ID, or "system")
    granted_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (role_id, capability)
);

```

Example data:

role_id granted_at	capability	granted_by
1207762798705123398 2026-07-09 20:00:00+00	players:write	system
1207762798705123398 2026-07-09 20:00:00+00	broadcast:send	system
1203226789569101894 2026-07-09 20:00:00+00	server:control	system

Behavior: A role with rows in this table gets exactly the capabilities listed. A role without rows falls back to its tier defaults. Admin and owner roles always get ALL capabilities regardless of this table.

6.2 Table: dune.rbac_audit_log

Records every capability-gated action with full actor identity and result. Used for SOC 2 compliance and incident investigation.

```
CREATE TABLE IF NOT EXISTS dune.rbac_audit_log (  
  id SERIAL PRIMARY KEY,  
  timestamp TIMESTAMPTZ DEFAULT now(),  
  actor_id TEXT NOT NULL,  
    -- "discord:143064109775060993" or "local:admin"  
  actor_name TEXT, -- "DarkDante" for readability  
  action TEXT NOT NULL, -- capability string:  
    "players:write"  
  target_type TEXT, -- "player", "server",  
    "backup", "config", "guild"  
  target_id TEXT, -- specific entity ID  
    (player_controller_id, etc.)  
  route TEXT, -- API path: "/api/players/1/  
    give-item"  
  result TEXT NOT NULL, -- "success", "denied", "error"  
  detail JSONB -- operation metadata  
    (redacted, no secrets)  
);  
  
CREATE INDEX idx_rbac_audit_timestamp ON dune.rbac_audit_log  
  (timestamp DESC);  
CREATE INDEX idx_rbac_audit_actor ON dune.rbac_audit_log  
  (actor_id, timestamp DESC);  
CREATE INDEX idx_rbac_audit_action ON dune.rbac_audit_log  
  (action, timestamp DESC);
```

What is NOT logged: Secrets, tokens, passwords, raw SQL results, internal IPs, stack traces, PII beyond actor identity. The detail JSONB field is redacted before storage.

7. Route Registration Pattern

7.1 Current Pattern

```
// server.js – if/else chain with regex path matching  
// NO capability enforcement  
if (path === "/api/players/:id/give-item" && req.method ===  
  "POST")  
  return giveItemRoute(req, res);  
if (path === "/api/server/status")  
  return commandJson(res, "status");
```

7.2 Target Pattern

Every Web UI API route declares its capability at registration time. This is enforced at **startup** — a missing or unknown capability panics the server immediately, not at request time. This pattern is modeled after dune-admin's `handleAPI()` function which panics on "" or unknown capabilities.

```
// New handleAPI function (server.js)
function handleAPI(method, pathPattern, capability, handler) {
  // 1. Validate capability exists – fail at startup, not at
  //    runtime
  if (!capability || !ALL_CAPABILITIES.has(capability)) {
    throw new Error(
      `handleAPI: ${method} $
        {pathPattern} registered with unknown capability "$
        {capability}". ` +
      `Valid capabilities: ${[...ALL_CAPABILITIES].join(", ")}`
    );
  }

  // 2. Record route->capability mapping for the permissions audit
  //    UI
  routeCapabilities.set(`${method}:${pathPattern}`, capability);

  // 3. Register the wrapped handler
  routes.push({
    method,
    pattern: pathPattern,
    handler: wrapWithCapabilityCheck(handler, capability)
  });
}

function wrapWithCapabilityCheck(handler, capability) {
  return async function(req, res) {
    const session = req.authSession;
    if (!session) {
      json(res, 401, { error: "Authentication required.", code:
        "unauthenticated" });
      return;
    }
    if (!session.capabilities.has(capability)) {
      auditAccessDenied(session, capability, req);
      json(res, 403, {
        error: `Not authorized. Required capability: $
          {capability}.`,
        code: "not_authorized"
      });
      return;
    }
    auditAccessAllowed(session, capability, req);
    return handler(req, res);
  };
}
```

```
};  
}
```

7.3 Migration Example

```
// Phase 2 migration – route registration  
handleAPI("GET", "/api/server/status",  
    STATUS_READ,    commandJsonRoute("status"));  
handleAPI("GET", "/api/server/readiness",  
    READINESS_READ, safeCommandJsonRoute("readiness"));  
handleAPI("GET", "/api/server/services",  
    SERVICES_READ,  commandJsonRoute("services"));  
handleAPI("GET", "/api/players",  
    PLAYERS_READ,   listPlayersRoute);  
handleAPI("POST", "/api/players/:id/give-item",  
    PLAYERS_WRITE,  giveItemRoute);  
handleAPI("POST", "/api/players/:id/specializations/*",  
    PLAYERS_WRITE,  specializationRoute);  
handleAPI("POST", "/api/players/:id/clean-inventory",  
    PLAYERS_DELETE, cleanInventoryRoute);  
handleAPI("GET", "/api/database/tables/:schema/:table",  
    DATABASE_READ,  tablePreviewRoute);  
handleAPI("POST", "/api/database/query",  
    DATABASE_QUERY, queryRoute);  
handleAPI("POST", "/api/backups/restore",  
    BACKUPS_MANAGE, restoreBackupRoute);
```

7.4 Startup Validation

```
// At server startup, after all routes are registered:  
function validateRouteCapabilities() {  
    const unregistered = [];  
    // Check each registered route has a capability  
    for (const route of routes) {  
        const key = `${route.method}:${route.pattern}`;  
        if (!routeCapabilities.has(key)) {  
            unregistered.push(key);  
        }  
    }  
    if (unregistered.length > 0) {  
        throw new Error(  
            `The following routes are missing capability declarations: $  
            {unregistered.join(", ")}. ` +  
            `Every API route must be registered via handleAPI(method,  
            path, capability, handler).`  
        );  
    }  
    console.log(`Route capability validation: $  
        {routeCapabilities.size} routes OK`);  
}
```

8. Capability Resolution Engine

8.1 resolveSessionCapabilities(session, db)

This function is called once during session creation. The result is stored in the session object and checked on every subsequent request.

```
function resolveSessionCapabilities(session, db) {  
  // 1. Local admin fallback → all capabilities  
  if (session.authSource === "local") return ALL_CAPABILITIES;  
  
  // 2. Owner role → all capabilities (bypass)  
  const ownerRoleIds =  
    parseCsv(process.env.DISCORD_OWNER_ROLE_IDS);  
  if (session.roleIds.some(r => ownerRoleIds.includes(r))) return  
    ALL_CAPABILITIES;  
  
  // 3. Admin role → all capabilities (bypass)  
  const adminRoleIds =  
    parseCsv(process.env.DISCORD_ADMIN_ROLE_IDS);  
  if (session.roleIds.some(r => adminRoleIds.includes(r))) return  
    ALL_CAPABILITIES;  
  
  // 4. DB-backed role→capability mapping  
  const dbCaps = db.query(  
    `SELECT capability FROM dune.rbac_role_capabilities WHERE  
      role_id = ANY($1)`,  
    [session.roleIds]  
  );  
  
  // 5. DB has entries → use them (skip tier fallback)  
  if (dbCaps.rows.length > 0) return new Set(dbCaps.rows.map(r =>  
    r.capability));  
  
  // 6. No DB entries → fallback to hardcoded tier matrix  
  const tier = resolveTier(session.roleIds);  
  return CAPABILITIES_BY_TIER[tier];  
}
```

8.2 resolveTier(roleIds)

```
function resolveTier(roleIds) {  
  const ownerIds = new  
    Set(parseCsv(process.env.DISCORD_OWNER_ROLE_IDS));  
  const adminIds = new  
    Set(parseCsv(process.env.DISCORD_ADMIN_ROLE_IDS));  
  const moderatorIds = new  
    Set(parseCsv(process.env.DISCORD_MODERATOR_ROLE_IDS));  
  const observerIds = new  
    Set(parseCsv(process.env.DISCORD_OBSERVER_ROLE_IDS));
```

```

    if (roleIds.some(r => ownerIds.has(r))) return "owner";
    if (roleIds.some(r => adminIds.has(r))) return "admin";
    if (roleIds.some(r => moderatorIds.has(r))) return "moderator";
    if (roleIds.some(r => observerIds.has(r))) return "observer";
    return "public";
}

```

8.3 Resolution Order (Priority)

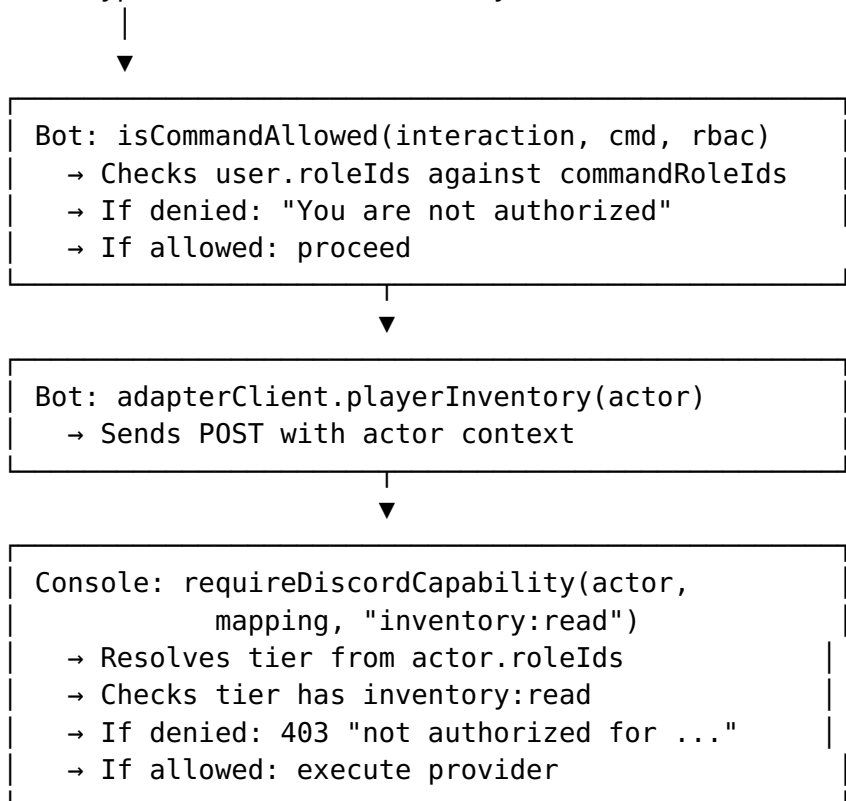
1. authSource === "local" → ALL_CAPABILITIES
 2. roleIds n DISCORD_OWNER_ROLE_IDS ≠ ∅ → ALL_CAPABILITIES
 3. roleIds n DISCORD_ADMIN_ROLE_IDS ≠ ∅ → ALL_CAPABILITIES
 4. rbac_role_capabilities has rows for role → Set(db rows)
 5. rbac_role_capabilities is empty →
CAPABILITIES_BY_TIER[tier]
-

9. Bot Integration

9.1 Dual-Layer Enforcement

The bot and console adapter form a belt-and-suspenders security model:

User types /dune data inventory



Both layers must pass. The bot gates command visibility (user doesn't see commands they can't use). The adapter gates execution (server-side enforcement even if the bot were compromised).

9.2 Bot Config Update

The bot's `config.js` currently has per-command `commandRoleIds` that list specific Discord role IDs for each command. After RBAC, commands map to capabilities instead, and the tier→capability matrix handles the rest:

```
// Before: per-command role list (fragile, 26 separate configs)
commandRoleIds: {
  "data:inventory": mergeRoleIds(moderatorRoleIds, adminRoleIds),
  "data:storage": mergeRoleIds(adminRoleIds),
}

// After: capability-based (one config per command)
commandCapabilities: {
  "data:inventory": "inventory:read",      // requires moderator+
  "data:storage": "storage:read",          // requires admin+
}
```

The `isCommandAllowed` function consults the capability mapping, resolves the user's tier, and checks if the tier includes the required capability. This eliminates 26 per-command role configurations in favor of one tier matrix.

10. Implementation Phases

Phase 1 — Extend Capability Model (1 session)

Files: `console/api/src/integrations/discord/policy.js`, `console/api/test/discordAdapter.test.js`

- Add 10 new capability constants to `DISCORD_CAPABILITIES`
- Extend `CAPABILITY_BY_TIER` with expanded capability sets
- Update `EXPERIMENTAL_READ_ONLY_CAPABILITIES` to exclude new write capabilities
- Update adapter test expectations for new capability counts
- **No breaking changes** — purely additive

Phase 2 — Web UI Route Registration (2 sessions)

Files: console/api/src/auth.js, console/api/src/server.js, console/api/src/config.js

- Add ALL_CAPABILITIES constant with all 20 capability descriptions
- Add resolveTier(), resolveSessionCapabilities(), CAPABILITIES_BY_TIER to auth.js
- Add handleAPI(method, path, capability, handler) to server.js
- Add wrapWithCapabilityCheck(), validateRouteCapabilities() to server.js
- Extend session object with capabilities: Set
- Migrate existing routes from if/else chains to handleAPI() calls
- Every route must have a capability or the server won't start

Phase 3 — Discord OAuth2 + DB-Backed Roles (3 sessions)

Files: console/api/src/auth.js, console/api/src/server.js, console/api/src/config.js, console/api/src/duneDb.js

- Add Discord OAuth2 flow endpoints:
 - GET /api/auth/discord/login — redirect to Discord
 - GET /api/auth/discord/callback — exchange code, create session
- Add DISCORD_OAUTH_* env var support to config.js
- Add OAuth2 state tracking with CSRF protection
- Create dune.rbac_role_capabilities table (auto-created on first use)
- Create dune.rbac_audit_log table (auto-created)
- Wire session enrichment with capabilities at login time
- Add RBAC admin API:
 - GET /api/rbac/capabilities — list all capabilities
 - GET /api/rbac/roles — list roles with capabilities
 - PUT /api/rbac/roles/:id — update role capabilities
- Local fallback: existing ADMIN_PASSWORD = owner tier
- Add audit helper: auditAccess(session, capability, target, result)

Phase 4 — Web UI Permissions Tab (2 sessions)

Files: console/web/src/features/rbac/, console/web/src/App.tsx

- New Permissions tab in sidebar navigation
- Capability matrix grid: roles (rows) × capabilities (columns) with checkboxes
- Audit log viewer: filterable table by actor, action, date range, result
- Admin-only access (auth:manage capability required to view/edit)

- Uses existing HeroUI component library

11. Migration Path

Step	Breaking?	Rollback
Add 10 capabilities (Phase 1)	No — additive	Revert commit
Refactor route registration (Phase 2)	No — routes work unchanged during migration; handleAPI is additive	Revert commit
Add OAuth2 + DB tables (Phase 3)	No — ADMIN_PASSWORD still works; DISCORD_OAUTH_CLIENT_ID not set = local only	Remove DISCORD_OAUTH_CLIENT_ID env var
DB table empty → tier fallback	No — empty table = existing hardcoded behavior	Delete rows = back to defaults
Permissions tab (Phase 4)	No — additive UI	Revert commit

Key backward compatibility guarantee: If DISCORD_OAUTH_CLIENT_ID is not set in the environment, the Web UI uses local password authentication — identical to current behavior. If dune.rbac_role_capabilities is empty (0 rows), the hardcoded tier matrix is used — identical to current behavior. Every phase is independently deployable and revertable. No existing installs break.

12. Key Decisions

#	Decision	Options Considered	Rationale
1	DB-backed roles from Phase 1	Hardcoded tiers first → DB later vs. DB from start	DB from start: operators can customize without code changes. Empty table = existing behavior. No migration needed later.

#	Decision	Options Considered	Rationale
2	Discord OAuth2 for Web UI	Tiered passwords (A), local accounts (B), Discord OAuth2 (C), API tokens (D)	Discord OAuth2: Same role IDs as the bot. No new accounts to manage. One configuration point for both bot and Web UI. Users already have Discord accounts.
3	Local admin fallback	No fallback vs. ADMIN_PASSWORD retains owner access	ADMIN_PASSWORD fallback: Emergency access when Discord is down. Server owners without Discord can still administer their server.
4	Owner/admin auto-capabilities	All roles require explicit assignment vs. owner/admin bypass	Bypass: Follows dune-admin convention. Server owners should never lock themselves out. No configuration needed for full access.
5	Startup panic on missing capability	Silently skip vs. runtime error vs. startup panic	Startup panic: Every route must be gated. Missing capability is a programmer error caught during development, not a security hole discovered in

#	Decision	Options Considered	Rationale
6	Belt-and-suspenders enforcement	Bot-only vs. adapter-only vs. both	production. Mirrors dune-admin's behavior. Both: The bot gates visibility (user sees only commands they can use). The adapter gates execution (server-side enforcement even if bot compromised). Defense in depth.
7	In-memory session capabilities	Per-request DB lookup vs. session-stored	Session-stored: Capabilities don't change mid-session. Avoids DB query on every request. Stale capabilities require re-login (12h max session).

13. Environment Configuration

13.1 Required Environment Variables

Variable	Required	Default	Description
DISCORD_OWNER_ROLE_IDS	Yes	—	Comma-separated owner-tier Discord role snowflake IDs
DISCORD_ADMIN_ROLE_IDS	Yes	—	Comma-separated admin-tier

Variable	Required	Default	Description
			Discord role snowflake IDs
DISCORD_MODERATOR_ROLE_IDS	Yes	—	Comma-separated moderator-tier Discord role snowflake IDs
DISCORD_OBSERVER_ROLE_IDS	Yes	—	Comma-separated observer-tier Discord role snowflake IDs

13.2 Optional Environment Variables

Variable	Required	Default	Description
DISCORD_OAUTH_CLIENT_ID	For Web UI OAuth2	—	Discord application client ID. If not set, Web UI uses local password only.
DISCORD_OAUTH_CLIENT_SECRET	For Web UI OAuth2	—	Discord application client secret
DISCORD_OAUTH_REDIRECT_URI	For Web UI OAuth2	—	OAuth2 callback URL (must match Discord app config)
ADMIN_PASSWORD	For local fallback	Auto-generated file	Local admin password. Used when Discord OAuth2 is unavailable or the user chooses local login.
ADMIN_AUTH_DISABLED	No	0	Set to 1 to skip all authentication

Variable	Required	Default	Description
			(development only)

13.3 Example .env

```
# Discord Role Mapping (shared with bot)
DISCORD_OWNER_ROLE_IDS=1203226789569101894
DISCORD_ADMIN_ROLE_IDS=1203226789569101894
DISCORD_MODERATOR_ROLE_IDS=1207762798705123398
DISCORD_OBSERVER_ROLE_IDS=1203226789569101894,1207762798705123398

# Discord OAuth2 (Web UI login)
DISCORD_OAUTH_CLIENT_ID=1516816812006969494
DISCORD_OAUTH_CLIENT_SECRET=<from-discord-developer-portal>
DISCORD_OAUTH_REDIRECT_URI=http://50.123.64.61:8088/api/auth/discord/callback

# Local fallback (optional – keeps existing ADMIN_PASSWORD behavior)
ADMIN_PASSWORD=<secure-random-password>
```

14. Sources & References

Source	Relevance
Icehunter/dune-admin — cmd/dune-admin/ auth_capabilities.go	Capability model design: 30 named capabilities, per-route registration with <code>handleAPI()</code> , startup panic on missing capability, owner bypass, DB-backed role mapping. Our 20-capability model is a subset adapted for our Node.js architecture.
Discord OAuth2 Documentation	OAuth2 authorization code grant flow: <code>/authorize</code> , <code>/oauth2/token</code> , <code>/users/@me</code> , <code>/users/@me/guilds</code> endpoints. State parameter for CSRF protection. identify and guilds scopes.
Red-Blink/dune-awakening-selfhost-docker	Upstream core repository. Our fork syncs with this. Reference for existing auth model, session management, and route structure.
yacketrj/dune-awakening-selfhost-docker	Our core fork. Branch integration/discord contains all current work. <code>console/api/src/auth.js</code> is the existing authentication module to extend. <code>console/api/src/</code>

Source	Relevance
<u>yacketrj/dune-awakening-selfhost-discordbot</u>	integrations/discord/policy.js is the existing capability model to expand. Discord bot repository. src/commands.js contains isCommandAllowed() and executeDuneCommand(). src/config.js contains per-command role allowlists to migrate to capability-based.
<u>Semantic Versioning 2.0.0</u>	Versioning convention for capability API changes.
<u>OWASP Session Management Cheat Sheet</u>	Session security best practices: HttpOnly cookies, SameSite=Lax, CSRF tokens for mutations, 12h rotation.
<u>SOC 2 Common Criteria</u>	Audit trail requirements guiding the rbac_audit_log schema: actor identity, action, target, result, timestamp, route.

Document generated 2026-07-09. Branch: *integration/discord*. All capabilities, tiers, and schema designs are proposed and subject to implementation validation.